

Classes & Memory

Advanced Programming
Brent van Bladel



Classes

A class is user-defined type providing an abstraction to a concept.

- Group data that forms a larger concept together.
- Link functionality to data.

```
1.  class Coordinate {
2.      float _x;
3.      float _y;
4.      public:
5.          Coordinate(float x, float y) : _x{x}, _y{y} { };
6.          ~Coordinate() { };
7.
8.          void print() {
9.              cout << _x << ", " << _y << endl; }
10. };
11.
12. int main() {
13.     Coordinate c{5, 10};
14.     c.print();
15. }
```

Class vs Object

Class: Abstraction of a concept. Blueprint of a type.

- **Attribute**
- **Operations**

Object: Instance of a class. Exists only at runtime.

- **State**
- **Behaviour**

Class vs Object

Class: Abstraction of a concept. Blueprint of a type.

- **Attribute:** (data member) Type and range of possible values.
- **Operations:** (member functions) Procedure that can be applied on the attributes.

Object: Instance of a class. Exists only at runtime.

- **State:** The values of the members at a given time.
- **Behaviour:** How the operation acts given the current state.

Class vs Object

Class: Abstraction of a concept. Blueprint of a type.

- **Attribute:** (data member) Type and range of possible values.
- **Operations:** (member functions) Procedure that can be applied on the attributes.

Object: Instance of a class. Exists only at runtime.

- **State:** The values of the members at a given time.
- **Behaviour:** How the operation acts given the current state.

You can only define a class once.

You can have any number of objects of a class.

Memory Layout of a Class

```
1.  class Coordinate {  
2.      float _x;  
3.      float _y;  
4.      public:  
5.          Coordinate(float x, float y) : _x{x}, _y{y} { };  
6.          ~Coordinate() { };  
7.          void print() { cout << _x << ", " << _y << endl; }  
8.  };  
9.  
10. int main() {  
11.     Coordinate c{5, 10};  
12.     c.print();  
13. }
```

STACK

...	0	0	5	10	0	0	0	0	...
-----	---	---	---	----	---	---	---	---	-----

Memory Layout of a Class

```
1.  class Coordinate {  
2.      float _x;  
3.      float _y;  
4.      public:  
5.          Coordinate(float x, float y) : _x{x}, _y{y} { };  
6.          ~Coordinate() { };  
7.          void print() { cout << _x << ", " << _y << endl; }  
8.  };  
9.  
10. int main() {  
11.     Coordinate c{5, 10};  
12.     c.print();  
13. }
```

STACK

...	0	0	5	10	0	0	0	0	...
-----	---	---	---	----	---	---	---	---	-----

STATIC
(ROM)

...	Coordinate()	~Coordinate()	print()	...
-----	--------------	---------------	---------	-----

Constructors

A **constructor** is an explicit means to initialize an object.

A constructor is called on object-creation to make sure it starts in a valid state.

```
1. class Coordinate {  
2.     float _x;  
3.     float _y;  
4.     public:  
5.         Coordinate(float x, float y) : _x{x}, _y{y} { };  
6.         ~Coordinate() { };  
7. };
```


Default Constructor

A **default constructor** initializes an object without any given data (e.g. constructor without parameters).

```
1. class Coordinate {  
2.     float _x;  
3.     float _y;  
4.     public:  
5.         Coordinate() : _x{0}, _y{0} { };  
6.         Coordinate(float x, float y) : _x{x}, _y{y} { };  
7.         ~Coordinate() { };  
8.     };
```

Implicit Default Constructor

A **default constructor** will be implicitly created if there are no user-defined constructors.

```
1. class Coordinate {  
2.     float _x;  
3.     float _y;  
4.     public:  
5.         ~Coordinate() {};  
6. };
```

Implicit Default Constructor

As soon as there is one constructor defined, there will be no implicit default constructor generated!

```
1.  class Coordinate {  
2.      float _x;  
3.      float _y;  
4.      public:  
5.          Coordinate(float x, float y) : _x{x}, _y{y}{};  
6.          ~Coordinate() {};  
7.  };  
8.  
9.  int main() {  
10.      Coordinate c; // error, no default constructor  
11.  }
```

Default keyword

Ask the compiler to generate a constructor or destructor.

```
1.  class Coordinate {
2.      float _x;
3.      float _y;
4.      public:
5.          Coordinate() = default;
6.          Coordinate(float x, float y) : _x{x}, _y{y}{};
7.          ~Coordinate() = default;
8.  };
9.
10. int main() {
11.     Coordinate c; // ok, default constructor generated
12. }
```

In-Class Initializers

Data members can be initialized in the class definition, which will be used if the member is not initialized in the called constructor.

```
1. class Coordinate {  
2.     float _x{0.0};  
3.     float _y{0.0};  
4.     public:  
5.         Coordinate() = default;  
6.         ~Coordinate() = default;  
7. };
```

Member Initializer Lists

Data members can be initialized via an initializer list instead of in the constructor body.

```
1. class Coordinate {  
2.     float _x;  
3.     float _y;  
4.     public:  
5.         Coordinate(float x, float y): _x{x}, _y{y} { };  
6.         ~Coordinate() = default;  
7. };
```

Member Initializer Lists

```
1. class Circle{  
2.     float _radius;  
3.     Coordinate _center;  
4.     public:  
5.         Circle(float r, Coordinate c){  
6.             _radius = r; _center = c;  
7.         };  
8.         ~Circle() = default;  
9.     };
```

```
1. class Circle{  
2.     float _radius;  
3.     Coordinate _center;  
4.     public:  
5.         Circle(float r, Coordinate c) : _radius{r}, _center{c} { };  
6.         ~Circle() = default;  
7.     };
```

Member Initializer Lists

```
1. class Circle{  
2.     float _radius;  
3.     Coordinate _center;  
4.     public:  
5.         Circle(float r, Coordinate c){  
6.             _radius = r; _center = c;  
7.         };  
8.         ~Circle() = default;  
9.     };
```

Four function calls to initialize _center

```
1. class Circle{  
2.     float _radius;  
3.     Coordinate _center;  
4.     public:  
5.         Circle(float r, Coordinate c) : _radius{r}, _center{c} { }  
6.         ~Circle() = default;  
7.     };
```

Three function calls to initialize _center

Member Initializer Lists

```
1. class Circle{
2.     float _radius;
3.     Coordinate _center;
4.     public:
5.         Circle(float r, Coordinate c){
6.             _radius = r; _center = c;
7.         };
8.         ~Circle() = default;
9.     };
```

Four function calls to initialize _center:

1. Copy Constructor for c
2. Default constructor for _center
3. Assignment operator on _center
4. Destructor for c

```
1. class Circle{
2.     float _radius;
3.     Coordinate _center;
4.     public:
5.         Circle(float r, Coordinate c) : _radius{r}, _center{c} { }
6.         ~Circle() = default;
7.     };
```

Three function calls to initialize _center

Member Initializer Lists

```
1. class Circle{
2.     float _radius;
3.     Coordinate _center;
4.     public:
5.         Circle(float r, Coordinate c){
6.             _radius = r; _center = c;
7.         };
8.         ~Circle() = default;
9.     };
```

Four function calls to initialize _center:

1. Copy Constructor for c
2. Default constructor for _center
3. Assignment operator on _center
4. Destructor for c

```
1. class Circle{
2.     float _radius;
3.     Coordinate _center;
4.     public:
5.         Circle(float r, Coordinate c) : _radius{r}, _center{c} { }
6.         ~Circle() = default;
7.     };
```

Three function calls to initialize _center:

1. Copy Constructor for c
2. Copy Constructor for _center
3. Destructor for c

Copy Constructor

A **copy constructor** initializes an object given a different object of the same class.

Signature: X(const X&)

```
1. class Coordinate {  
2.     float _x;  
3.     float _y;  
4.     public:  
5.         Coordinate(float x, float y) : _x{x}, _y{y} { };  
6.         Coordinate(const Coordinate& other) : _x{other._x}, _y{other._y} { };  
7.         ~Coordinate() = default;  
8. };
```

Copy Constructor vs Assignment Operator

A copy constructor constructs and initializes memory.

An assignment operator assigns a state to a previously constructed object.

```
1. class Coordinate {  
2.     float _x, _y;  
3.     public:  
4.         Coordinate(float x, float y) : _x{x}, _y{y} {};  
5.         Coordinate(const Coordinate& other) : _x{other._x}, _y{other._y} {};  
6.         Coordinate& operator=(const Coordinate& other) {  
7.             _x = other._x;  
8.             _y = other._y;  
9.             return *this;  
10.        }  
11.        ~Coordinate() = default;  
12.    };
```

Explicit keyword

```
1.  class Coordinate {
2.      float _x, _y;
3.      public:
4.          explicit Coordinate(float a) : _x{a}, _y{a} { };
5.          ~Coordinate() = default;
6.  };
7.
8.  class Circle {
9.      float _radius{1.0};
10.     Coordinate _center;
11.     public:
12.         Circle(Coordinate c) : _center{c} { };
13.  };
```

Mark constructor to not implicitly convert types.

1. Circle c1{5}; // error
2. Circle c2{Coordinate{5}}; // ok

Bracket initialization vs parentheses

```
1. class Coordinate {  
2.     float _x{0.0};  
3.     float _y{0.0};  
4. };
```

```
1. int main() {  
2.     Coordinate coor{};  
3. }
```

```
1. int main() {  
2.     Coordinate coor();  
3. }
```

Bracket initialization vs parentheses

```
1. class Coordinate {  
2.     float _x{0.0};  
3.     float _y{0.0};  
4. };
```

```
1. int main() {  
2.     Coordinate coor{};  
3. }
```

Object creation.

```
1. int main() {  
2.     Coordinate coor();  
3. }
```

Compiled as a function declaration!
“Most Vexing Parse”

Rule of thumb: always use braced initialization.

Bracket initialization vs parentheses

Rule of thumb: always use braced initialization.

Exception: vector initialization

1. `vector<int> v1(10);` // vector of 10 elements with default value 0
2. `vector<int> v2{10};` // vector of 1 element with value 10
- 3.
4. `vector<int> v3(1, 2);` // vector of 1 element with default value 2
5. `vector<int> v4{1, 2};` // vector of 2 elements with values 1 and 2

Destructors

A **destructor** is an explicit means to destroy an object.

A destructor is called on object-deletion to make sure it ends in a valid state.

```
1. class Circle {  
2.     float _radius{1};  
3.     Coordinate* _center;  
4.     public:  
5.         explicit Circle(Coordinate c) : _center{new Coordinate{c}} {};  
6.         ~Circle() { delete _center;}  
7. };
```

Destructors

A **destructor** is called explicitly using *delete* for objects on the heap and implicitly at the end of scope for objects on the stack.

```
1.  int main() {  
2.      Coordinate* c_on_heap = new Coordinate{5, 10};  
3.      Coordinate c_on_stack{5, 10};  
4.  
5.      delete c_on_heap; // destructor called on heap  
6.  
7.  } // end of scope: destructor called on stack
```

Memory & Classes in C++

Quiz

Which functions gets called?

```
1. int main() {  
2.     Coordinate c;  
3. }
```

Which functions gets called?

```
1.  int main() {  
2.      Coordinate c;  
3.  }
```

Answer: Default constructor
Destructor

Which functions gets called?

```
1. int main() {  
2.     Coordinate c{5, 10};  
3. }
```

Which functions gets called?

```
1. int main() {  
2.     Coordinate c{5, 10};  
3. }
```

Answer: Constructor
Destructor

Which functions gets called?

```
1. int main() {  
2.     Coordinate c1;  
3.     Coordinate c2{c1};  
4. }
```


Which functions gets called?

```
1.  int main() {  
2.      Coordinate c1;  
3.      Coordinate c2{c1};  
4.  }
```

Answer:

Default Constructor (c1)

Copy Constructor (c2)

Destructor (c2)

Destructor (c1)

Which functions gets called?

```
1. int main() {  
2.     Coordinate c1;  
3.     Coordinate c2 = c1;  
4. }
```

Which functions gets called?

```
1. int main() {  
2.     Coordinate c1;  
3.     Coordinate c2 = c1;  
4. }
```

Answer:

Default Constructor (c1)

Copy Constructor (c2)

Destructor (c2)

Destructor (c1)

Which functions gets called?

```
1.  int main() {  
2.      Coordinate c1;  
3.      Coordinate c2;  
4.      c2 = c1;  
5.  }
```

Which functions gets called?

```
1.  int main() {  
2.      Coordinate c1;  
3.      Coordinate c2;  
4.      c2 = c1;  
5.  }
```

Answer:

Default Constructor (c1)

Default Constructor (c2)

Assignment Operator (c2)

Destructor (c2)

Destructor (c1)

Which functions gets called?

```
1. int main() {  
2.     Coordinate c = {5, 10};  
3. }
```

Which functions gets called?

```
1.  int main() {  
2.      Coordinate c = {5, 10};  
3.  }
```

Answer:

Constructor

Destructor

Which functions gets called?

```
1.  int main() {  
2.      Coordinate c = Coordinate {5, 10};  
3.  }
```


Which functions gets called?

```
1.  int main() {  
2.      Coordinate c = Coordinate {5, 10};  
3.  }
```

Answer:

Constructor

Destructor

Which functions gets called?

```
1. int main() {  
2.     Coordinate c1;  
3.     Coordinate c2 = Coordinate{c1};  
4. }
```

Which functions gets called?

```
1.  int main() {  
2.      Coordinate c1;  
3.      Coordinate c2 = Coordinate{c1};  
4.  }
```

Answer:

Default Constructor (c1)

Copy Constructor (c2)

Destructor (c2)

Destructor (c1)

Tip: Add prints to the different functions to see what gets called!

```
1. class Coordinate {
2.     float _x, _y;
3.     public:
4.         Coordinate() : _x{0}, _y{0} { cout << "default constructor" << endl; };
5.         Coordinate(float x, float y) : _x{x}, _y{y} { cout << "constructor" << endl; };
6.         Coordinate(const Coordinate& other) : _x{other._x}, _y{other._y} { cout << "copy constructor" << endl; };
7.
8.         Coordinate& operator=(const Coordinate& other) {
9.             cout << "assignment" << endl;
10.            _x = other._x;
11.            _y = other._y;
12.            return *this;
13.        }
14.        ~Coordinate() {cout << "destructor" << endl; };
15.    };
```